

FUNDAMENTOS Y APLICACIONES DE BLOCKCHAINS

Homework 4

Depto. de Computación, UBA, 2do. Cuatrimestre 2025

1/11/25

Student:

Due: 13/11/25, 17:00 hs

Instructions

- Upload your solution to Campus; make sure it's only one file, and clearly write your name on the first page. Name the file '<your last name>_HW4.pdf.'

If you are proficient with $\text{\LaTeX}$, you may also typeset your submission and submit in PDF format. To do so, uncomment the "`%\begin{solution}`" and "`%\end{solution}`" lines and write your solution between those two command lines.

- Your solutions will be graded on *correctness* and *clarity*. You should only submit work that you believe to be correct.
- You may collaborate with others on this problem set. However, you must **write up your own solutions** and **list your collaborators and any external sources (including ChatGPT and similar generative AI chatbots)** for each problem. Be ready to explain your solutions orally to a member of the course staff if asked.

This homework contains 4 questions, for a total of 40 points.

1. **Bitcoin backbone: Chains of variable difficulty.**

(a) (5 points) Describe Bahack's "difficulty raising" attack.

Solution: El ataque de Bahack consiste en que el adversario fuerza la dificultad de la red para que sea muy alta, inhabilitando a los mineros honestos.

El adversario va a minar en privado **toda una epoch** con timestamps falsos (ya que sería imposible que mine en un tiempo tan corto). Luego, la red la recibe, y siendo la cadena más larga, la va a aceptar. Luego, la dificultad de la próxima epoch va a ser muy alta, dado que estos bloques se minaron en un tiempo muy corto. Entonces, la próxima epoch tendría una dificultad muy alta, y los mineros honestos no van a poder minar bloques válidos en un tiempo razonable.

(b) (5 points) What aspect of Bitcoin's target recalculation function makes the attack ineffective? Elaborate.

Solution: La solución a este problema en la red de bitcoin es que el *target* tiene un límite de cambios; es decir, no puede cambiar demasiado entre dos epochs. La regla es que $T' \in [\frac{T}{4}; 4T]$, siendo T el *target* de la epoch actual y T' el *target* de la próxima epoch.

Esto permite que la red no tenga cambios tan abruptos en dificultad, y vuelve el ataque de Bahack mucho menos efectivo.

2. Proofs of Stake (PoS).

- (a) (5 points) Consider the *long range attack* on a PoS blockchain in a permissionless environment. Assume that an honest-stake majority always holds, and that all the parties are aware of the global clock. Describe the scenarios wherein the honest parties lose their advantages, and explain why freshly joining parties cannot distinguish an honest chain from other chains by the longest-chain selection rule.

Solution:

El problema que tiene un nodo que acaba de entrar a la red es que no puede verificar si las cadenas que recibe son honestas o no. Esto es porque, aunque tenga acceso al reloj global, no puede verificar que los timestamps de la cadena que recibe sean los correctos (por ejemplo, no podría saber si el timestamp de 6 meses antes es realmente de hace 6 meses o de hace 5 minutos, creado por un atacante). Solamente podría verificar que no sean timestamps futuros.

Hay otro problema relacionado con un nuevo participante, y es el siguiente. Pongamos un caso de un adversario que tenía mucho stake (digamos en el bloque 10, y que ahora estamos en el 100) pero que luego lo vendió, puede crear una cadena alternativa más larga (digamos 130 bloques) en la cual él sigue teniendo todo ese stake. Como él sigue teniendo todo ese stake, la cadena sería creíble para un minero que acaba de entrar. Notar que este ataque no le cuesta nada a un atacante, puesto que crear bloques no es computacionalmente caro como en esquemas de *PoW*.

- (b) (5 points) A PoS protocol performs leader election based on the stakes each party owns. Recall that the initial stake distribution is hard-coded in the genesis block. Does a PoS protocol assume a PKI? Further, the stakes in a party's account have monetary values. Where can the initial stakes come from?

Solution: No, un protocolo de PoS no necesita una PKI. Esto es porque cada participante se identifica por su dirección, que es justamente su clave pública. Entonces no hace falta una autoridad central que nos asegure que una clave pública pertenece a un participante, porque la clave pública es justamente lo que identifica a un participante, ya que con una firma digital podemos verificar que el participante (identificado por su clave pública) es el dueño de las claves privadas.

Para decidir de donde vienen los stakes iniciales hay varias opciones. Una muy común es que en el bloque génesis se distribuya el stake entre los participantes que son fundadores de la red.

Otra opción es que la cadena empiece como PoW y luego se vuelva PoS. Esto es lo que sucedió con Ethereum 2.0. En este caso, el stake inicial serían lo que

los mineros ya habían obtenido con la red PoW.

También puede ser que se "quemen" otras monedas a cambio de stake en esta nueva cadena, para demostrar que se está asumiendo un riesgo.

3. Verifiable Random Functions (VRFs).

- (a) (4 points) Enumerate the security properties a VRF should satisfy.

Solution: Una VRF debe cumplir que, para un par (VK, SK) fijo, y un input X , se cumpla que produce un valor único, pseudoaleatorio y verificable. La parte de único quiere decir que nadie puede encontrar un Y' y una prueba π' tal que $\text{Verify}(VK, X, Y', \pi') = 1$. Luego, que sea verificable nos permite saber que es algo que fue generado solamente por quien se identifica por la clave pública. Es decir, que valga que si $(Y, \pi) = \text{Eval}(SK, X)$, entonces $\text{Verify}(VK, X, Y, \pi) = 1$. La pseudoaleatoriedad implica no solo que sea impredecible, si no que es indistinguible de una secuencia de bits verdaderamente aleatoria.

- (b) (6 points) We stated in class that given a hash function, modeled as a *random oracle*, and an unforgeable signature scheme, VRFs are readily realizable. Show that that's indeed the case by proposing a construction and arguing its security—i.e., it achieves the desired security properties. Follow the VRF terminology we used in class.

Solution:

Dado que tenemos un oráculo aleatorio y un esquema de firma digital, podemos construir una VRF de la siguiente manera. Asumimos que estamos mirando desde la perspectiva del firmante con claves (SK, VK)

El objetivo es generar el número aleatorio Y para un input X . No podemos simplemente usar $H(X)$ porque cualquier otro podría hacer lo mismo. Tampoco podemos hacer $H(SK, X)$, ya que, aunque otros no podrían replicarlo, no tendrían manera de verificarlo. Entonces generamos una prueba π , que esta vinculada a X y solo la puede producir el dueño de la SK . $\pi = \text{Sign}(SK, X)$, donde Sign es la firma que forma parte del "unforgeable signature scheme". Es determinista.

Luego de generar esa prueba, que es impredecible para otros y está vinculada al valor de X , necesitamos que sea aleatoria. Entonces, la pasamos por el oráculo, haciendo $Y = H(\pi)$.

Luego, se publica (X, Y, π) , y vemos que es verificable porque todos los demás pueden chequearlo con $\text{Verify}(VK, X, Y, \pi)$, siendo VK la clave pública del firmante.

Veamos que esto cumple las propiedades de seguridad.

- Verificable: ya vimos el camino que tiene que seguir un participante que quiere verificar la autenticidad del mensaje X

- Unicidad: Esto nos lo garantiza el "unforgeable signature scheme", que es determinista. Un adversario no va a poder encontrar (Y', π') , con $Y \neq Y'$ que haga verdadero el predicado *Verify*, dado que la única manera va a ser que $\pi = \pi'$ (pues la firma es determinista), y eso solo va a ser posible si $Y = Y'$.
- Pseudoaleatoriedad: "Unforgeable signature scheme" nos garantiza que es imposible para un adversario (sin SK) generar una firma válida π' para un mensaje X .

4. The Ouroboros protocol.

- (a) (5 points) We saw in class that in the Ouroboros protocol the slot leader election process is abstracted out and modeled as an “ideal functionality.” Describe one realistic approach to elect the slot leader, and explain why there are $\perp$ symbols in the characteristic strings.

Solution: El objetivo del leader election es que cada participante pueda saber si fue elegido para liderar el slot.

Por cada ronda (“slot”), cada participante quiere saber si fue elegido. Para esto, genera un π con su clave privada y un input público y reconocible (ej: el id del slot). Luego, calcula $Y = H(\pi)$. Si sucede que $Y \leq T_i$ (donde T_i es el target de la ronda para el participante i , ya que debería ser proporcional a la cantidad de stake que tiene cada uno, si no todos tendrían la misma probabilidad de ser elegido), entonces ese participante es el líder de esa ronda, y todos los demás pueden verificar que así lo es, con $\text{Verify}(\text{VK}, X, Y, \pi)$.

Este es un esquema simple, que comparte con el protocolo de bitcoin el hecho de generar un hash que tiene que ser menor a un target. Pero que la probabilidad de acertar no depende de la cantidad de veces que se intenta, si no de la cantidad de stake que tiene cada participante. Es más, el intento es único por ronda, ya que cada participante solo puede generar un π por cada input X .

El símbolo $\perp$ en la *characteristic string* nos indica que no hay líder para esa ronda. Esto puede suceder perfectamente si ninguno de los participantes logra generar un $Y \leq T_i$ para su slot. Entonces, se declara que no hay líder para esa ronda.

- (b) (5 points) Describe the implementation of the *dynamic* stake setting. Why can’t the parties use the most recent stake distribution (i.e., the stake distribution at the end of the previous epoch)?

Solution: La implementación del stake dinámico se basa en que el stake se puede ir transfiriendo entre participantes. Es crucial definir cómo tomamos la información de esta distribución, ya que es necesaria para ver quiénes lideran el slot.

En un modelo estático esta distribución se basa en el stake inicial del bloque génesis.

Lo que hace el protocolo de ouroboros es tomar la distribución de una época anterior (no la inmediatamente anterior), mediante un modelo más complejo que distribuye el cómputo entre los participantes.

No se puede tomar la distribución justo al final de la época anterior, porque es un bloque **demasiado reciente**, por lo tanto puede haber forks que no

hayan sido resueltos aún. Entonces no se podría lograr un consenso de esta distribución entre todos los participantes. Además, previene que un atacante pueda hacer cambios en los bloques que publica para poder tener mayor stake en la época siguiente.

